Information Storage and Retrieval (IR)

Executive Midterm Study Guide & Algorithmic Blueprint

Course: IR | Scope: Ch 1, Lect 2, Lect 3, Lect 5 | Inverted Index | Skip Pointers | Stemming & Normalization

01 Introduction to Information Retrieval & Inverted Indexes (Chapter 1)

Formal Definition of Information Retrieval (IR)

Information Retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).

1.1 Information Need vs. Query

- **Information Need:** The semantic topic or desire for knowledge in the user's mind (e.g. "I want to know if drinking coffee raises blood pressure").
- **Query:** What the user actually types into the search engine to communicate that need (e.g. `coffee blood pressure`). Often ambiguous or incomplete.

1.2 Information Retrieval vs. Traditional Databases

Dimension	Traditional Database (RDBMS)	Information Retrieval (IR)
Data Structure	Structured relational tables with well-defined schemas.	Unstructured or semi-structured free-form text.
Semantics & Matching	Clear, deterministic semantics; exact boolean matching.	Ambiguous natural language; relevance ranking and probabilistic scoring.
Query Language	Formal declarative SQL statements.	Natural language keywords, boolean operators, or free-text queries.
Result Set	Exact set of rows matching the WHERE clause (unordered unless ORDER BY).	Ranked list of documents ordered by estimated relevance.

1.3 The Term-Document Incidence Matrix & Its Fatal Limitation

- **Incidence Matrix:** A 2D binary matrix where rows represent vocabulary terms (V) and columns represent documents (N). Cell $(t, d) = 1$ if term t appears in document d, and 0 otherwise.
- **The Sparsity Problem:** In a collection of $N = 1,000,000$ documents and $V = 500,000$ terms, the matrix contains $500,000 imes 1,000,000 = 500 billion$ cells! Over 99.9% of these cells are 0.

Storing this dense matrix is completely impractical.

1.4 The Inverted Index: The Standard Data Structure

An **Inverted Index** stores only the non-zero entries (the occurrences), mapping each term to the documents where it appears.

- **Dictionary (Vocabulary):** A sorted list of unique terms, typically held in RAM (using a B-Tree or Hash Table) for fast lookup. Includes the *Document Frequency (df)*.
- **Postings Lists:** For each term, a contiguous variable-length array or linked list of Document IDs (DocIDs) where the term occurs, sorted in ascending DocID order.

1.5 Inverted Index Construction Pipeline

1. **Document Parsing:** Extract raw character streams from formats (HTML, PDF, DOCX) and identify character encoding (UTF-8).
2. **Tokenizer:** Break character stream into a sequence of word tokens.
3. **Linguistic Modules:** Apply case folding, stop word handling, and stemming/lemmatization to produce canonical *terms*.
4. **Indexer:** Extract (Term, DocID) pairs, sort alphabetically by term and secondarily by DocID, and aggregate identical terms into dictionary entries with attached postings lists.

02 Boolean Query Processing, Optimization & Phrase Queries (Lecture 2)

2.1 The Postings Merge (Intersection) Algorithm

To evaluate a boolean AND query (e.g. `Brutus AND Caesar`), the search engine simultaneously traverses the sorted postings lists of both terms using two pointers (p_1, p_2).

```
Algorithm INTERSECT(p1, p2): answer = [] while p1 != NIL and p2 != NIL: if DocID(p1) == DocID(p2): answer.append(DocID(p1)) p1 = p1.next p2 = p2.next else if DocID(p1) < DocID(p2): p1 = p1.next else: p2 = p2.next return answer
```

> **Time Complexity**
>
> Because both lists are strictly sorted in ascending order of DocID, each pointer traverses its list at most once. The time complexity is strictly linear: **O(x + y)**, where x and y are the lengths of the two postings lists.

2.2 Query Optimization for Boolean Queries

- **Optimal AND Evaluation Order:** Always process terms in order of **increasing Document Frequency (df)** (i.e. shortest postings lists first).
- **Concrete Example:**
  - `Calpurnia` ($df = 4$)
  - `Brutus` ($df = 20,000$)
  - `Caesar` ($df = 40,000$)
  - Evaluate as: `(Calpurnia AND Brutus) AND Caesar`. The intermediate result of the first intersection contains at most **4 documents**! Intersecting 4 DocIDs with Caesar's list requires negligible time.

2.3 Phrase Queries: Biword vs. Positional Indexes

A standard inverted index only records document presence, not word order. It cannot distinguish `"Stanford University"` from a document containing "Stanford" on page 1 and "University" on page 50.

Index Approach	How It Works	Advantages & Disadvantages
Biword Index	Every adjacent pair of consecutive words is indexed as a discrete vocabulary term (e.g. "friends romans", "romans countrymen").	**Pros:** Fast lookup for 2-word phrases. **Cons:** Vocabulary explosion (V^2 potential pairs), false positives for longer phrases, cannot handle proximity queries.
Positional Index	For every term and document, stores the exact sequential word token position offsets: `term -> docID: `.	**Pros:** Solves arbitrary N-word phrase queries and proximity queries (`/k`). Standard industry solution. **Cons:** Postings lists expand 2–4× (occupying 35–50% of raw text volume).

Combination Scheme (Hybrid)	Biwords for highly frequent common phrases (e.g. "New York", "Michael Jackson"); positional index for arbitrary queries.	Optimal balance between throughput and storage efficiency.

2.4 Proximity Queries (`/k` Operator)

A proximity query like `employment /3 federal` requires that "employment" and "federal" occur within at most 3 words of each other ($|pos_1 - pos_2| \le 3$). The positional merge checks matching DocIDs and verifies position differences.

03 Faster Postings Merges & Skip Pointers (Lecture 3)

3.1 Skip Pointers: Mechanics & Motivation

When intersecting two postings lists where one is very short and the other is very long (or both are long), linear scanning inspects many DocIDs that could never match. **Skip Pointers** augment postings lists with forward shortcut pointers to jump ahead.

> **The Skip Decision Rule**
>
> During merge of p_1 and p_2: If p_1 has a skip pointer and **skip(p1) ≤ DocID(p2)**, take the jump directly to `skip(p1)`! If `skip(p1) > DocID(p2)`, do NOT take the skip (to avoid overshooting); advance p_1 by one step normally.

3.2 Placement of Skip Pointers: The Heuristic Rule of Thumb

- **The Tradeoff:** More frequent skips provide more jump opportunities but increase index storage size and pointer-checking overhead; fewer skips have low overhead but offer few chances to jump.
- **Heuristic Rule:** For a postings list of length P, place skip pointers evenly spaced every √**P** elements!
- **Example:** If a postings list contains $P = 10,000$ elements, place skips every $\sqrt(10, 0) = 100$ elements, resulting in roughly 100 evenly spaced skip pointers.
- **Dynamic Index Limitation:** Skip pointers work best on static indexes. In dynamic collections with frequent inserts and deletes, skip intervals degrade, requiring costly pointer maintenance.

04 The Term Vocabulary & Text Normalization (Lecture 5)

4.1 Document Parsing & Tokenization Challenges

- **Character Encodings:** Detecting whether raw bytes are UTF-8, UTF-16, or ASCII is vital to avoid mojibake (corrupted characters).
- **Tokenization:** The process of segmenting a stream of characters into discrete tokens (words).
  - *Hyphens:* Compound words (`Hewlett-Packard`, `state-of-the-art`, `co-education`) challenge whether to index as one token or split.
  - *Apostrophes:* Contractions (`aren't`) vs. possessives (`children's`) vs. proper names (`O'Neill`).
  - *Numbers & Formats:* Dates, model numbers (`Boeing 747`, `iPhone 15`), and chemical formulas have high discriminatory value and must be preserved.
  - *Language Specifics:* Chinese/Japanese have no whitespace delimiters, requiring morphological segmenters or character N-grams. Arabic has clitics (prefixes like 'and' و, 'the' ال) attached directly to nouns.

4.2 Stop Words: Traditional vs. Modern Paradigm

- **Stop Words:** Extremely common functional grammatical words (e.g. `the`, `a`, `is`, `of`, `to`, `at`) that appear in virtually all documents.
- **Traditional Approach:** Filter out 30 to 150 (or up to 750) stop words from the index to save disk space.
- **Modern Search Engine Approach:** Keep all or almost all stop words in the index. Reasons: (1) Storage compression has made index size manageable; (2) Users frequently search for exact phrases consisting heavily of stop words (e.g. `"To be or not to be"`, `"Let it be"`, `"The Who"`).

4.3 Normalization: Case Folding & Equivalence Classes

- **Equivalence Classes:** Mapping different surface tokens to a single canonical term (e.g. `U.S.A.` and `USA` → `usa`).
- **Case Folding:** Lowercasing all characters. While standard, it risks conflating proper nouns with common words (`US` country vs. `us` pronoun; `Bush` president vs. `bush` plant; `CAT` scan vs. `cat` animal).
- **Soundex:** Phonetic hashing algorithm based on consonant sounds, matching names with different spellings that sound identical (e.g. `Smith` and `Smyth`).

4.4 Stemming vs. Lemmatization (Crucial Exam Concept)

Dimension	Stemming (e.g. Porter Stemmer)	Lemmatization
Methodology	Crude, fast, heuristic suffix chopping without grammar or vocabulary lookup.	Full morphological analysis using vocabulary, grammar rules, and Part of Speech (POS) context.

Output	Often produces an artificial, non-word stem (e.g. `automate, automatic, automation` → `automat`).	Always produces a true, grammatically valid base dictionary form called a **lemma**.
Context Sensitivity	Blind to grammatical context; treats every word identically.	Context-aware: reduces `saw` to `see` if used as a verb, or leaves as `saw` if used as a noun. Reduces `better` to `good`.
Speed & Complexity	Very fast, low memory footprint, simple algorithmic rules.	Computationally intensive, requires extensive lexical dictionaries and grammatical parsers.

4.5 The Porter Stemmer (5 Phases) & Stemming Errors

- **Porter's Algorithm (1980):** The standard English stemmer consisting of 5 sequential phases of suffix stripping rules based on syllable measure m (vowel-consonant sequences).
- **Over-stemming Error:** Chopping too much, causing semantically distinct words to merge to the same stem (e.g. `universe` and `university` → `univers`; `police` and `policy` → `polic`). Hurts retrieval *precision*.
- **Under-stemming Error:** Failing to merge forms that share the same conceptual root (e.g. `adhere` and `adhesion`). Hurts retrieval *recall*.
- **Heaps' Law:** Vocabulary size V grows sublinearly with collection size N following $V = k N^Ξeta$ (where $Ξeta Ξpprox 0.4 - 0.6$), showing vocabulary continues to expand indefinitely as new documents are indexed.